

InternTables Design

COOL Compiler — `common/internTables.h` / `internTables.c`

Why InternTables is needed at all

A single generic `Table` only knows how to intern raw text. The compiler, however, needs to intern three semantically different kinds of text — identifiers/type names, string literal constants, and integer literal constants — and each kind is consumed differently by later phases (identifiers are pure lookup keys; string and integer constants must additionally become real objects emitted into the final assembly output). `InternTables` exists to bundle exactly the three tables the compiler needs, under one clearly-scoped, honestly-named struct, rather than scattering three independent global `Table*` variables through the codebase.

Naming discipline: this struct is deliberately **not** called "SymTable" or "symbol table". That name is reserved elsewhere for two different things: (1) the scoped lookup table built during semantic analysis (name → type, per scope), and (2) the actual linker/loader-facing symbol table written into a compiled object file. Reusing the name for the interning bundle would create exactly the kind of ambiguity this project has worked to avoid throughout.

Built on top of the generic Table

`InternTables` is a thin composition layer, not a reimplementation. It holds three independent `Table*` instances and nothing else:

```
typedef struct InternTables {
    Table *idTable;    /* OBJECTID and TYPEID text */
    Table *strTable;   /* STR_CONST text */
    Table *intTable;   /* INT_CONST text */
} InternTables;
```

Every actual hashing, bucket management, and collision handling is delegated entirely to `table.c`. `internTables_add_id`, `internTables_add_string`, and `internTables_add_int` are each a one-line call into `table_add_entry`, differing only in which of the three tables they target and what entry size they request. This keeps the generic table fully reusable (see `tables.pdf`) while giving the compiler a COOL-specific, purpose-named API to call.

Who owns InternTables

Exactly the same ownership rule as the underlying tables: the driver (`main.c`) creates `InternTables` once, before lexing begins, via `internTables_create()`, and destroys it once, via `internTables_destroy()`, after every phase that needs it (lexer, and eventually parser/semant/codegen) has finished. No individual phase creates, resizes, or frees any of the three underlying tables directly — each phase only ever receives an `InternTables*` pointer and calls its published `internTables_add_*` functions.

- This is deliberately the same shape of decision made for the tables themselves: the resource needs a lifetime spanning multiple, otherwise-unrelated phases, so ownership sits one level above all of them — at the driver — rather than being assigned to whichever phase happens to use it first.
- A smaller, narrowly-scoped struct (only the three tables, nothing else) was chosen over a single catch-all "CompilerContext" holding everything the compiler might ever need to share. A box should be named after exactly what is inside it; growing it into a junk drawer of unrelated fields (error lists, settings, etc.) later would make its name stop being trustworthy.

Why new entry types were introduced (StringEntry / IntEntry)

The base Entry struct (str, len, index, next) is sufficient for IdTable: an identifier is only ever used as a comparison key, never emitted as data. String and integer constants are different: at code-generation time, each unique constant must become an actual runtime object in the compiled program (a COOL String or Int object), so their entries eventually need extra codegen-specific behaviour — producing an assembly reference to themselves, and producing their own object definition — that an IdEntry would never use.

```
typedef struct StringEntry {
    Entry base;    /* must be first field */
} StringEntry;

typedef struct IntEntry {
    Entry base;    /* must be first field */
} IntEntry;
```

Rather than adding fields to the shared base Entry that only two of its three consumers would ever use, each specialised kind gets its own struct that *wraps* the shared base. This keeps IdTable lean (it never pays for fields it does not need) while still letting StringTable and IntTable grow additional codegen-only metadata later (e.g. an "already emitted" flag) without touching the generic table implementation at all.

How inheritance was simulated in C, and why it works

C has no language-level inheritance, but the same effect can be achieved through **struct embedding**: placing the base struct as the very first member of the derived struct. The C standard guarantees that a struct's first member is located at offset zero within the struct — meaning a pointer to the derived struct and a pointer to its first member share the exact same address.

```
StringEntry *se = ...;
Entry *base = (Entry*) se;    /* always valid: same address as se->base */
StringEntry *back = (StringEntry*) base; /* valid to cast back, same reasoning */
```

This is precisely why it was safe for table_add_entry to be written once, generically, in terms of plain Entry*: it can malloc a block of entry_size bytes (which may be sizeof(Entry), sizeof(StringEntry), or sizeof(IntEntry), depending on the caller) and treat the first bytes of that block as a valid Entry, regardless of what extra bytes for a derived struct follow it. The caller is then free to cast the returned Entry* back to StringEntry*/IntEntry* when it needs the fuller type.

- This technique is exactly the same one used by real C codebases that need inheritance-like polymorphism without a language feature for it (for example, GTK's object system).
- It requires the base struct to always be the *first* field, with no exceptions — if a derived struct ever placed other fields before the embedded base, the offset-zero guarantee would no longer hold and the cast would become undefined behaviour.
- It made possible the specific design goal set for this project: a single, generic `table_add_entry` that works uniformly across three tables whose entries have different eventual shapes, with the size difference communicated at the call site via a plain `size_t entry_size` parameter rather than the table needing any type-level awareness of `StringEntry` or `IntEntry` at all.

Summary of the ownership and data flow

Stage	What happens
main.c	Creates <code>InternTables</code> once, before lexing. Destroys it once, at the very end.
Lexer	Calls <code>internTables_add_id</code> / <code>add_string</code> / <code>add_int</code> as each token is recognised; stores the returned <code>Entry*</code> in the token's value field.
Parser / Semant	Compares tokens' <code>Entry*</code> pointers by identity; never re-creates or frees any table.
Codegen	Walks every entry in <code>strTable</code> and <code>intTable</code> (via their <code>StringEntry*/IntEntry*</code> view) to emit each unique constant's object definition exactly once.